[0x3f]

branchless mod 3

note.

rsi, rdi likely parameters/vars NOT stored on the stack

```
mov     rbx,3          rbx := 3 ~ 0011
mov     r8,rax         r8 := rax_0 (save x)
mov     rcx,rax        rcx := rax_0 - 1
dec     rcx
and     rax,rcx        rax = rax_0 ∧ (rax_0 - 1)
xor     edx,edx        edx := 0
div     rbx            rdx:rax / rbx → rax 2, remainder → rdx
mov     rsi,rdx        rsi := rdx ~ remainder (*)
mov     rax,r8         rax = rax_0
or      rax,rcx        rax = rax_0 ∨ (rax_0 - 1)
xor     edx,edx        edx := 0
div     rbx            rdx:rax / rbx → rax 2, remainder → rdx
inc     rdx            rdx = remainder(x∨(x-1)) + 1
cmp     rdx,rbx
sbb     rdi,rdi        rdi := -CF ∈ {0,-1}
and     rdi,rdx        rdi = {0,-1} ∧ remainder + 1
bsf     rax,r8         rax := 0-index of lsb of r8 (rax_0)
```

rax := x

clears lowest set bit (idiom)

rsi = ~~x∧(x-1)/3~~ remainder

=0 ⟺ no set bits above lsb

(x∧(x-1))/3 is set all bits above lsb % 3

every bit > lsb

every bit < lsb

let r = remainder (x∨(x-1))/3

rdi = { 0 r=2 ; -1 r=0,1 }

{1,2}

is set if bits above lsb % 3

rax

higher the above of the bit ???

0-index of lsb (x) is position (idx) of lowest set bit

ex) 0x40 ~ 0100 0000 = x

rsi = 2 = 0011

rdi = fff...f = -1

rax = 6 = 0110

[0x3f]

rbx := 3

r8 := x

rax = x-1

rax = x ∧ (x-1) → IGNORE (remainder of upper/3 is not > 2^31)

rdx := edx := 0

rax = ~~rdx~~ : rax / rbx is rax = (x ∧ (x-1))/3

rax (remainder ↑ rdx)

rsi := remainder ∈ rdx of

rax = x ∨ (x-1) → IGNORE (.. ∨)

rdx := edx := 0

rax = ~~rdx~~ : rax / rbx is rax = (x ∨ (x-1))/3

rax (remainder ↑ rdx)

rdx = remainder + 1

divisor

compare (remainder + 1 , ~~rbx~~ 3) is (CF = 1 ⟺) remainder + 1 ∧ ~~rbx~~ 3 (idiom)

rdi = -CF ∧ (remainder + 1)

= { 0 remainder = 2 ; -1 remainder = 0,1 } = { 0 ; r = 2 ; 1,2 ; r = 0,1 } ⇕

rax := 0-index of lsb of r8 (rax_0) (x) (remainder ∧ (3-1) = 2)

is rax = index of lsb (x)

is rsi ~ ∈ {0,1,2} summary of bits above lsb

~~rax~~ 0-index of position of lsb

mov rbx, 3 — rbx := 0011 ← same from later

mov r8, rax — r8 := rax_0

mov rcx, rax } → rcx := $rax_0 - 1$

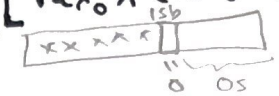
dec rcx

and rax, rcx — $rax = rax_0 \wedge rcx = rax_0 \wedge (rax_0 - 1)$

xor edx, edx — edx := 0

div rbx } → $rax = rax/3 = [rax_0 \wedge (rax_0 - 1)]/3$

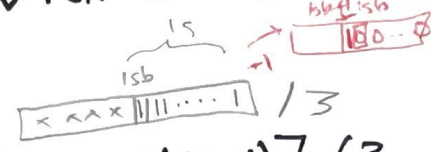
mov rsi, rdx — rsi := remainder$\{[rax_0 \wedge (rax_0 - 1)]/3\} \in \{0,1,2\}$ (rdx)



clear lowest set bit /3

mov rax, r8 } → $rax = r8 = rax_0 \vee rcx = rax_0 \vee (rax_0 - 1)$

or rax, rcx



xor edx, edx — edx := 0

div rbx } → $rax = [rax_0 \vee (rax_0 - 1)]/3$

(fill ≤ lsb/3) + 1

inc rdx } → rdx = remainder$\{[rax_0 \vee (rax_0 - 1)]/3\} + 1$ → adjusted remainder

cmp rdx, rbx — cmp(rdx, rbx = 3) is rdx < 3 ⇒ CF := 1; ELSE (rdx ≥ 3) ⇒ CF := 0

sbb rdi, rdi — $rdi = rdi - (rdi + CF) \xrightarrow{rdx \geq 3} 0$; $\xrightarrow{rdx < 3}$ 0xff...f

and rdi, rdx } → $rdi = rdi \wedge rdx \xrightarrow{rdx \geq 3} 0$; $\xrightarrow{rdx < 3}$ rdx

rdi = 0 OR rdx

ie

bsf rax, r8

rdx ≥ 3: rdi = $\{0 \wedge rdx \geq 3\} = 0$

branchless mod 3

rdx < 3: rdi = 0xff...f ∧ rdx < 3

rdi = rdx < 3

rax := bsf(r8 = rax_0 = 0011) = 0

least significant set bit 0-index } = 0 } ie zero out rax